유현종

백엔드 개발자

운영 데이터에서 문제를 찾아 원인까지 추적하고, 직접 세운 기준으로 검증해 개선을 끝내는 백엔드 개발자입니다.

경력
툰스퀘어 백엔드 개발자, 정규직
2025.04 – 2026.04

개인 프로젝트
KAIROS 기업용 AI 게이트웨이
2026.04 – 2026.09

연락처
guswhd903@naver.com
github.com/Drawjustin

핵심 성과

1,500ms → 50ms
요청이 몰릴 때 응답 시간, gap lock 원인 제거 실무

약 60만 건
6개월간 포인트 증감, 잔액·이력 대조 불일치 0건 실무

30일 → 즉시
QA 지원 도구로 없앤 확인 대기 시간 실무

167.8초 → 114ms
민감정보 검사의 ReDoS 취약점 제거 개인 프로젝트

2	**툰스퀘어 실무**	재현되지 않던 장애 진단, 포인트 정합성 설계, QA 지원 도구
3	**KAIROS 구조**	게이트웨이를 거쳐야만 AI에 닿는 업무망, 민감정보 처리
4	**KAIROS 검증**	몰릴 때·겹칠 때·멈출 때를 숫자로 다시 확인
5	**음성 민원 AI**	AI가 틀려도 사람이 바로잡을 지점을 둔 통화 흐름

일하는 방식

원인은 데이터로 좁힙니다
재현되지 않던 지연을 슬로우 쿼리와 접속량 대조로 좁혀 gap lock을 찾았습니다.

틀려도 되짚을 수 있게 남깁니다
포인트의 모든 증감을 두 함수와 이력으로 모아 실패하면 복원되게 했습니다.

동작한 뒤에도 다시 잽니다
기능 테스트를 통과한 게이트웨이를 다시 측정해 ReDoS 취약점을 찾았습니다.

기술

실무 TypeScript · Node.js · MySQL · GraphQL
교육·개인 Java · Kotlin · Spring Boot · JPA · PostgreSQL · Redis
인프라 AWS · Terraform · Docker · Prometheus · k6

교육과 자격

KT AIVLE SCHOOL 우수 수료 · SSAFY 수료
정보보안공학과 졸업 · 정보처리기사 · SQLD

운영 중인 장애는 데이터로 좁히고, 함께 바뀌는 값은 기록으로 지켰습니다

AI 웹툰 창작 서비스(투닝플러스)와 글로벌 콘텐츠 판매 서비스(Deev)의 서버 API와 관리자 기능을 개발·운영했습니다. 월 이용자 약 3천 명 규모입니다.

재현되지 않던 지연의 원인 진단 단독 수행

문제 요청이 몰릴 때만 지연·오류가 발생하고, 쿼리를 단독 실행하면 정상

원인 인덱스 없는 조건의 DELETE가 gap lock을 넓게 걸어 요청끼리 서로 대기

해결 실행 쿼리 조건에 맞춰 인덱스를 추가해 잠금 범위 축소

슬로우 쿼리 시각 → 접속량과 대조 → DB 프로세스 확인 → **gap lock 특정**

1,500ms → 50ms
요청이 몰릴 때 해당 구간 응답

5건 → 0건
락 대기 오류, 일 평균

포인트 정합성 설계와 복구 경로 사수와 2인 설계



- 경로가 늘어도 잔액은 두 함수로만 바뀌고, 사유·담당자를 이력으로 기록
- 외부 AI 호출 실패 시 차감 내역대로 복원, 복원도 실패하면 알림 발송
- 동시 요청은 잠금으로 이중 차감 방지

약 60만 건
6개월간 처리한 포인트 증감

불일치 0건
이력 합계와 잔액을 주기적으로 쿼리로 대조

QA 지원 도구로 확인 대기 제거 먼저 제안

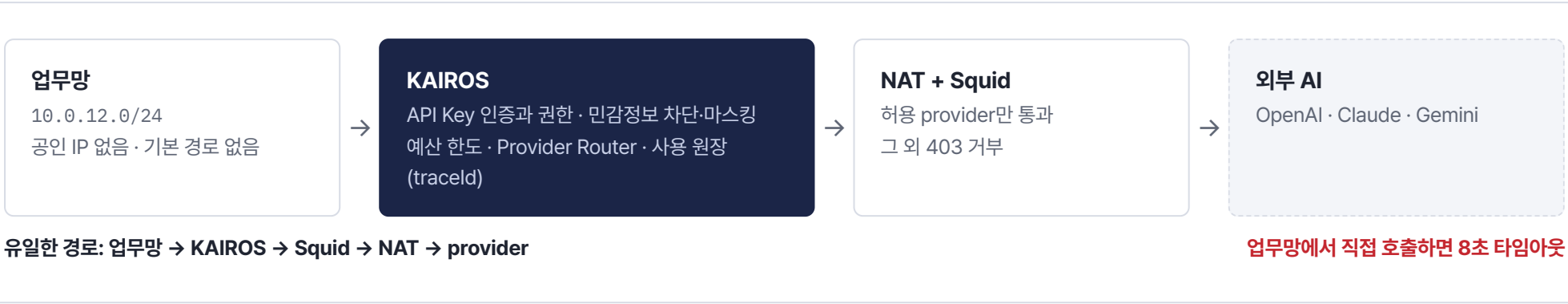
QA 담당자 한 명이 두 서비스를 맡으며 재가입 제한(30일), 포인트 만료(배치)처럼 시간이 지나야 확인되는 항목이 밀렸습니다. 날짜 수정과 계정 탈퇴를 관리자 기능으로 만들되, 만료일 수정은 실서비스 배치와 같은 단계를 거치게 하고 운영 환경에서는 막았습니다.

30일 → 즉시
가장 오래 기다려야 했던 확인

제가 지킨 기준 · 증상만 보고 짐작으로 자원부터 늘리지 않습니다. 언제 생기는지를 데이터로 좁혀 원인을 확인한 뒤 고치고, 함께 바뀌는 값은 틀렸을 때 되짚을 수 있게 기록을 남깁니다.

업무망에서는 게이트웨이를 거쳐야만 AI에 닿도록 만들었습니다

사내에서 AI를 쓰되 기밀이 외부로 나가지 않게 하는 것이 목표입니다. 커밋 95건, 테스트 137건 · github.com/Drawjustin/KAIROS



외부 경로가 구조적으로 없습니다

```
# VPC Flow Log, 업무망 발신
119건 전부 VPC 내부
외부 0건, REJECT도 0건
```

규칙으로 막았다면 REJECT 기록이 남습니다. 라우팅 테이블에 외부 경로 자체를 두지 않았고, 프록시는 별도 인스턴스에 두어 설정 하나로 우회되지 않게 했습니다.

민감정보는 보내기 전에 처리합니다

```
주민번호 포함 → 403, provider 미호출
010-1234-5678 → 010-****-5678
```

주민·카드번호 등은 차단, 전화·이메일은 마스킹해서 provider에 보냅니다. 로그에는 원문 없이 종류·건수만 남깁니다.

개발 AI도 같은 관문을 지납니다



모델이 고른 도구는 실행 직전에 허용 목록과 다시 대조합니다. 모델의 판단이 곧바로 사내 시스템 접근이 되지 않게 하기 위해서입니다.

제가 지킨 기준 · 막아야 할 것은 규칙보다 구조로 막습니다. 규칙은 잘못 고치는 순간 뚫리지만, 처음부터 없는 길은 뚫릴 수 없습니다. AI를 막지 않으면서도 무엇이 오갔는지는 남게 했습니다.

기능 테스트를 통과한 뒤에도, 몰릴 때·겹칠 때·멈출 때를 다시 쟀습니다

모든 요청이 지나는 계층이라, 보호 장치가 오히려 서비스를 멈추게 하지 않는지 숫자로 확인했습니다.

입력이 길어질 때

167.8초 → 114ms

요청 1건(상한 20,000자 메시지 50개)의 민감정보 검사 시간



원인 이메일 정규식에 앞쪽 경계 검사가 빠져 검사 시간이 입력 길이의 제곱으로 증가 (ReDoS)

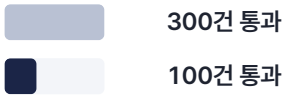
상한 허용된 메시지 1건(20,000자)만으로도 3.3초 → 수정 후 2ms

조치 경계 검사와 백트래킹 상한 추가, 회귀 테스트로 고정

요청이 동시에 겹칠 때

정확히 100건

예산 한도 100, 동시 요청 300건에서 통과한 수



문제 조회 후 갱신하면 300건이 모두 한도를 통과했는데 사용량은 1건만 기록됨 (lost update)

조치 판정과 증가를 조건부 UPDATE 한 쿼리로 처리

부하가 걸리고 외부가 멈출 때

오버헤드 약 30ms

200 VUs에서 성공 응답 p95 3.03초, mock 지연 3초를 뺀 게이트웨이 몫



격리 provider별 동시 호출 32건, 초과분은 대기 없이 즉시 거절해 처리 중인 요청의 지연을 지킴

장애 OpenAI 장애 주입 시 서킷이 열리고, 허용된 모델 안에서만 Claude로 전환

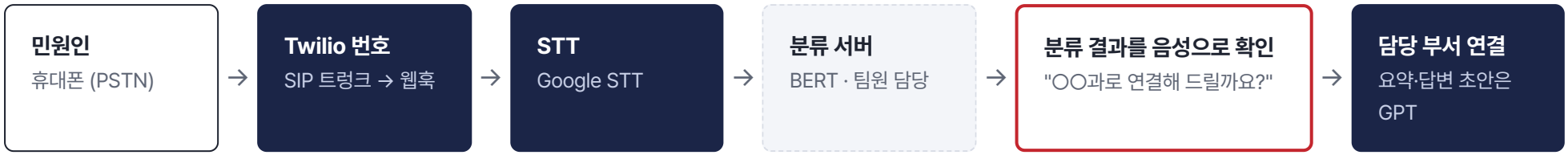
제가 지킨 기준 · 동작하는 것을 내놓은 시점이 아니라, 된다는 것을 숫자로 보일 수 있는 시점을 끝으로 봅니다. 찾은 결함은 고치는 데서 끝내지 않고 회귀 테스트로 고정합니다.

측정 2026-09-24, JDK 21, 8코어, 예열 후 중앙값 · 부하는 mock provider 3초 고정 지연, 로컬 k6 · 장애 주입 2026-09-09

AI가 틀려도 민원인이 처음부터 다시 설명하지 않도록 설계했습니다

현직 공무원 인터뷰로 확인한 전화 응대 부담을 줄이는 시스템입니다. 저는 모델 학습을 제외한 통화 파트의 백엔드·인프라를 맡았습니다.

내 담당: 통화 파트



아니오: 다시 말하기 · 분류 서버 무응답: 기본값으로 통화 유지

점선은 팀원 담당 · 답변 최종 판단은 공무원

통신사 제약은 목적을 바꿔 풀었습니다

문제 통신사 최소 회선 요건과 비용이 프로젝트 규모에 맞지 않음

판단 필요한 것은 통신사 계약이 아니라 음성을 받아오는 경로

해결 회선을 SaaS로 제공하는 Twilio로 전화를 웹훅 이벤트로 수신

STT는 목적에 맞는 수준으로

비교 클로바·구글·Twilio에 같은 음성을 넣어 전사 결과를 직접 비교

기준 분류의 입력이라 완벽한 전사보다 정확도 대비 단가가 중요

선택 팀 논의를 거쳐 Google STT

AI가 틀릴 때 바로잡을 지점

확인 분류 결과를 음성으로 확인받고, 다르면 다시 말하게 함

장애 분류 서버가 응답하지 않아도 기본값으로 통화 유지

역할 두 장치 모두 팀에 제안해 채택, 통화 파트에서 직접 구현

최종 시연 전화 한 통으로 **접수부터 답변 초안까지** 동작 · 분류 서버를 중단해도 **통화 유지**

제가 지킨 기준 · 무슨 일이 생기든 통화는 끊지 않는다는 기준 하나를 먼저 세웠습니다. AI는 흐름을 돕는 부분이지 흐름 자체가 아니어서, AI가 틀리거나 멈춰도 사람이 바로잡고 흐름은 이어지게 설계합니다.